



National Institute of Applied Sciences and Technology

CARTHAGE UNIVERSITY

Graduation Project

Specialty: **SOFTWARE ENGINEERING**

Your project's title

Presented by

First LAST

INSAT Supervisor : **Mr. LAST First**

Company Supervisor : **Ms. LAST First**

Presented on: --/--/20XX

JURY

Mr. President LAST (President)

Ms. Reviewer LAST (Reviewer)

Academic Year: 20XX/20YY

Acknowledgements

Thank you all!

Table of Contents

List of Figures	iii
List of Tables	iv
Abstract	v
General Introduction	1
I Theoretical Study	3
1 State of the Art	3
2 Study of the Existing System	3
II Design	6
1 Recommendations	6
2 Diagrams	6
2.1 Sequence Diagram	7
2.2 Class Diagram	7
III Implementation	9
1 Tools and Languages	9
2 Application Walk-Through	9
2.1 Table Example	9
2.2 Code Example	9
3 Notes on the Bibliography	10
Conclusion and Perspectives	12
References	13
Appendix: Miscellaneous remarks	14

List of Figures

I.1	State of the art	4
II.1	Stereotypes	7

List of Tables

III.1 Comparison table	10
----------------------------------	----

Abstract

This is the English abstract of your project. It must be longer and presented in more detail than the abstract you write on the back of your report.

General Introduction

Writing a good computer-science project report [1] follows a few universal rules. Everyone writes with their own voice and signature, but some conventions remain shared [2].

The Table of Contents is the first thing a reviewer reads. It should be:

- Detailed enough¹ – three numbering levels usually suffice;
- Balanced across chapters, except for the Introduction and the Conclusion which are naturally shorter than the others;
- Built from descriptive titles. Avoid generic “Design”; prefer “Design of the management application for ...”. Slightly longer titles are better than an impersonal summary.

An introduction should be written as well-crafted paragraphs. It usually has four parts:

1. **Context** – the general domain of your application (web, BI, enterprise software, etc.);
2. **Problem statement** – what needs in this context motivate your project;
3. **Contribution** – a short description of your application, without going into implementation details. The introduction introduces the work, it does not summarize it;
4. **Report outline** – the different chapters and what they cover. These items do not need to be numbered; prefer well-connected paragraphs.

¹But not too detailed

Part I

Part 1

Chapter I

Theoretical Study

Summary

1	Recommendations	6
2	Diagrams	6
2.1	Sequence Diagram	7
2.2	Class Diagram	7

Introduction

A theoretical study [3] typically contains one or both of the following parts.

1 State of the Art

This is a detailed survey of what already exists on the market or in the literature (hence the name “state of the art”). The goal is to compare existing solutions, analyse the result of that comparison, and explain why none of them fully address the problem at hand.

2 Study of the Existing System

This is usually performed when extending an existing piece of software or modifying an existing application. The study describes what is already in place in your working environment.

Conclusion

The conclusion is usually unnumbered and does not appear in the table of contents.



Figure I.1 – State of the art

Part II

Part 2

Chapter II

Design

Summary

1	Tools and Languages	9
2	Application Walk-Through	9
2.1	Table Example	9
2.2	Code Example	9
3	Notes on the Bibliography	10

Introduction

The design chapter is ***not always required***. When the work is a theoretical study or a system deployment, drawing class or sequence diagrams is unnecessary – often counter-productive. For development work, however, design is mandatory.

1 Recommendations

A few general rules:

- Pick a methodology – a unified process, an agile method, etc.

2 Diagrams

Choose the diagrams that are relevant to your application. The use-case, class and sequence diagrams are usually required. You may add any other diagram you find pertinent; for a multi-tier application, for instance, the deployment diagram is useful.

- Diagrams must be clear, readable and well explained – without drowning the reader in detail. Lengthy explanations get tiresome;
- If a diagram is too large, split it into several diagrams or abstract away some details. If splitting is impossible, print it on A3 and fold the page. The key requirement is that every label remains legible.

2.1 Sequence Diagram

A sequence diagram:

- Represents one possible scenario inside a use case. You do not need to show every possible execution path;
- Represents the interaction between objects – every instance in a sequence diagram should correspond to a class that appears in the class diagram;
- Many sequence diagrams are usually possible; pick two or three of the most important to include in the report (authentication is not one of them).

2.2 Class Diagram

A class diagram:

- Must reflect the chosen software architecture. If you use MVC, the three layers should be visible in the class diagram via packages;
- Stereotypes are strongly encouraged. For a web application, consider using the stereotypes in Figure II.1:



Figure II.1 – Stereotypes

- Don't confuse classes with database tables – resist the temptation to put an `id` field everywhere.

Conclusion

A short summary of the chapter, plus a brief lead-in to the next one.

Chapter III

Implementation

Introduction

This chapter covers the implementation work and the bibliography.

1 Tools and Languages

The technical study may live in this chapter, or be performed in parallel with the theoretical study (as the 2TUP model suggests). The goal here is to argue for your technology choices: a short state of the art with a comparison, a synthesis, and a justified choice of tools is expected.

2 Application Walk-Through

It is tempting to fill this section with every screenshot of the application you worked so hard on – resist. You should include screenshots, but pick them carefully and present them as a scenario: choose one execution path, say creating a new customer, then walk through the interfaces involved while briefly describing the application’s behaviour. Not too many images, not too much prose – concise, as always.

Avoid pasting raw code: nobody wants to read the contents of your classes. You may include snippets to highlight specific features [2, 3] when truly useful. Installation steps, configuration files and longer code excerpts belong in the appendix.

2.1 Table Example

You can use a tabular comparison between existing works. See Table [III.1](#).

2.2 Code Example

A Java snippet with syntax highlighting – see Listing [III.1](#).

Table III.1 – Comparison table

	Col1	Col2	Col3	Col4
Row1		X		
Row2	X			
Row3	X	X	X	X
Row4	X		X	X
Row5	X		X	X
Row6	X		X	X
Row7	X		X	
Row8	X	X	X	

Listing III.1 – Hello World in Java

```
public class HelloWorld {  
    // entry point  
    public static void main(String[] args) {  
        System.out.println("Hello, World");  
    }  
}
```

3 Notes on the Bibliography

Your bibliography must meet a number of criteria; otherwise it will get flagged (and sometimes flagged even when you think it shouldn't – everyone has a personal opinion on what makes a good bibliography).

- A bibliography in a good report should contain more books and articles than websites – it's a bibliography after all. Prefer peer-reviewed works for definitions, rather than jumping to Wikipedia;
- Entries are usually ordered alphabetically, or by theme (and alphabetically within each theme);
- A bibliography entry generally takes this form:

- A unique identifier – either a number [1] or the first author’s name plus year, e.g. [Kuntz, 1987];
- If a book: author(s), title, publisher, ISBN/ISSN, publication date;
- If an article: author(s), title, journal or conference, publication date;
- If a website or electronic document: title, URL, date accessed;
- If a thesis: author, title, defending university, year, page count;
- Examples:
 - [Bazin, 1992] BAZIN R., REGNIER B. Antiviral treatments and their clinical trials. Rev. Prat., 1992, 42, 2, p. 148–153.
 - [Anderson, 1998] ANDERSON P.JF. Checklist of criteria used for evaluation of metasites. [online]. University of Michigan, USA.
<http://www.lib.umich.edu/megasite/critlist.html>. (Accessed 1998-09-11.)
- In the body of the report, every claim taken from a reference must cite its identifier right after the claim. Failing to do so is plagiarism.

Conclusion

That’s it.

Conclusion and Perspectives

This is one of the most important and yet most neglected parts of the report. What you should not do here is talk about how rewarding the internship was, how it taught you to integrate into a team, what the working world is like, and so on.

Frankly, nobody cares – at least not in this section. That kind of material belongs in the Acknowledgements or in the Dedication.

A conclusion is simple: first, it summarizes what you described in the report – your contribution, the tools you used, your methodology. You can also mention the difficulties you encountered. Second, it lays out perspectives: features that could be added to your application, directions for future improvement.

References

- [1] LILIA SFAXI AND SOUHEIB YOUSFI. *How to write a good report*. Math-CS Department, INSAT (2015). [1](#)
- [2] MR. LATEX. Getting Started with LaTeX. <https://www.latex-project.org/>, (2008). [Online; accessed 2026-05-20]. [1](#), [9](#)
- [3] SOUHEIB YOUSFI AND LILIA SFAXI. *LaTeX Report*. Math-CS Department, INSAT (2015). [3](#), [9](#)

Appendix: Miscellaneous remarks

- A report must always be properly paginated.
- Prefer no more than two colours, and avoid fancy display fonts.
- Keep your report sober and professional.
- Avoid first-person singular pronouns; use the editorial “we”.
- If there is no section 1.2, do not introduce a 1.1.
- ALWAYS, ALWAYS have someone else (ideally outside the field) proofread your report for spelling and grammar.
- Always highlight your contribution – it should stand out clearly.
- Every chapter must contain an introduction and a conclusion.
- Keep a clear narrative thread throughout the report: the reader should follow a clean line of reasoning and see how the parts connect.
- Define abbreviations the first time they appear. If there are many, use a glossary.
- When describing the hardware environment, don’t talk about the laptop you developed on – that’s irrelevant. Mention only the hardware that affects the application itself. Whether you developed on Windows or Ubuntu does not matter.
- Never end a page with a heading.
- Define abbreviations on first use; if you have many, add a glossary.
- Keep sentences short – clear and concise is the rule.

Remember that your report is the face of your work: a bad report can overshadow excellent work. Pay it the attention it deserves.


